



APIs with Lambda + API Gateway



Seye Sanya

API Gateway > APIs > Resources - UserRequestAPI (8btuodpzwj) > Create method

Method details

Method type
GET

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.

☐ **HTTP**
Integrate with an existing HTTP endpoint.

☐ **Mock**
Generate a response based on API Gateway mappings and transformations.

☐ **AWS service**
Integrate with an AWS Service.

☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.

☒ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Response transfer mode [Info](#)

☒ **Buffered**
Wait to receive the complete response before beginning transmission.

☐ **Stream**
Send portions of the response without waiting for the complete response.

Lambda function



Introducing Today's Project!

In this project, I will demonstrate how to set up an API using AWS lambda and Amazon API Gateway. I'm doing this project to learn how APIs work and also, set up a snazzy logic tier in our three-tier architecture.

Tools and concepts

Services I used were Amazon API Gateway and AWS Lambda. Key concepts I learnt include Lambda functions, API resources and methods, Lambda proxy integration, writing API documentation, API stages, invoke URLs for an API.

Project reflection

This project took me approximately an hour and a half including the demo and secret time mission. The most challenging part was understanding and writing the Lambda function. It was most rewarding to create the resources and methods, and it was very informative.

I chose to do this project today because I wanted to learn more about the logic tier of a web app, in instances that explain how the code is run, relatively about the 'backend', or brains of my web app. The project definitely met my goals and I learnt a lot about how API works along the way. Something that would make learning with NextWork even better is including an AI chatbox to help vet errors from screenshots and the work as it is been done.



Lambda functions

AWS Lambda is a service that lets me run code without needing to manage any computers or servers personally. It is because of this that it is called a serverless service or Function as a service (FaaS). I'm using Lambda in this project to run code that helps me retrieve data

The code I added to my function will grab a userID from a triggered event, for instance, by submitting a userID through a form or field in a website, and queries for a piece of data in DynamoDB that matches usedID. The code also takes care of error handling.

```
JS index.mjs (in file) ↔ index.mjs (in Lambda code) ✓ ↶ ↷ ↵ ↴ 🔍 📖 ☐ ...  
JS index.mjs > ...  
1 1 import individual components from the DynamoDB client  
2 2 import { DynamoDBClient } from "@aws-sdk/client-dynamodb"  
3 3 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb"  
4 4  
5 5 const ddbClient = new DynamoDBClient({ region: 'us-east-1' });  
6 6 const ddb = DynamoDBDocumentClient.from(ddbClient);  
7 7  
8 8 async function handler(event) {  
9 9     const userId = event.queryStringParameters.userId;  
10 10     const params = {  
11 11         TableName: 'UserData',  
12 12         Key: { userId }  
13 13     };  
14 14  
15 15     try {  
16 16         const command = new GetCommand(params);  
17 17         const { Item } = await ddb.send(command);  
18 18         if (Item) {  
19 19             return {  
20 20                 statusCode: 200,  
21 21                 body: JSON.stringify(Item),  
22 22                 headers: { 'Content-Type': 'application/json' }  
23 23             };  
16 16         }  
17 17     } catch (error) {  
18 18         console.error(error);  
19 19         return {  
20 20             statusCode: 500,  
21 21             body: JSON.stringify({ message: 'Internal server error' })  
22 22         };  
23 23     }  
}
```

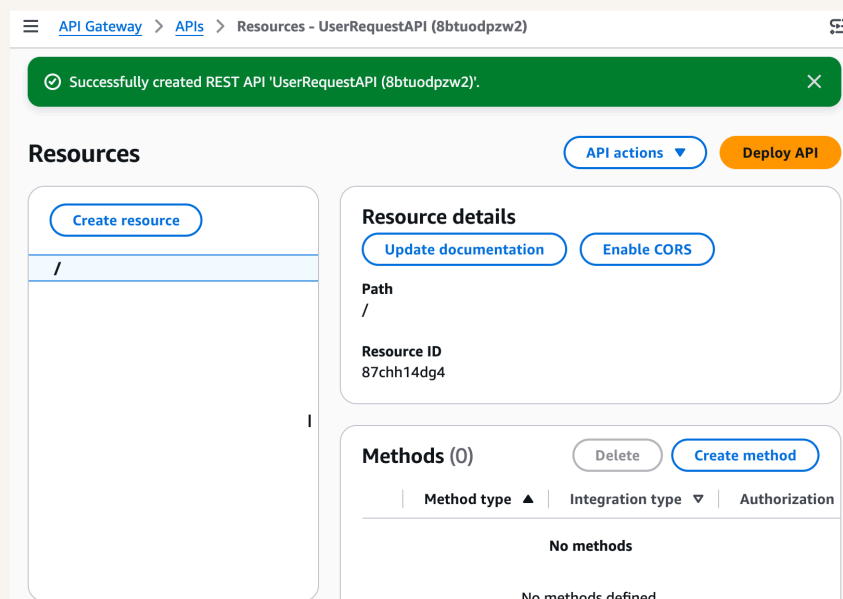


API Gateway

APIs are tools that enable communication between systems. There are different types of APIs, like HTTP APIs, REST APIs or Websocket APIs. My API is REST API which is compactible with lambda and many programming languages.

Amazon API Gateway is an AWS service that makes it easy for developers to create, publish, maintain, monitor, and secure APIs at any scale. I'm using API Gateway in this project to connect my web app user with my lambda function - the serverless backend

When a user makes a request, like clicking on a "Get User Data" button for example. API Gateway's role is to recieve the traffic and determine whether or not it is authorized, and passes it through to lambda if it is. API Gateway is also a traffic manager for requests.





API Resources and Methods

An API is made up of resources, which are different sections or different parts of the same API. For example, an API could have different sections for retrieving user data vs retrieving messaging data vs retrieving product data in the same app.

Each resource consists of methods, which are based on standard HTTP methods, which are different commands that let you interact with data over the internet.

I created a GET method that will connect the API Gateway with my lambda function. This means that when an end user makes a request with our API, for example "api.com/users", the API knows to pass the event or information if the user is retrieving data.





API Deployment

When you deploy an API, you deploy it to a specific stage. A stage is a snapshot of your API used to manage different API versions. I deployed to a production stage, also known as prod, which is where there is live traffic and users can access it.

To visit my API, I visited the Invoke URL for the API that is in production. The API displayed an error because the API is connected to a Lambda function that does not have permissions to DynamoDB, which may have triggered authentication issues, and I am yet to set up our DynamoDB table.

The screenshot shows the 'Deploy API' modal in the AWS console. The modal has a title bar with a close button (X). Below the title, there is a descriptive text: 'Create or select a stage where your API will be deployed. You can use the deployment history to revert or change the active deployment for a stage. [Learn more](#)'. The modal contains three main sections: 1. 'Stage' section with a dropdown menu currently showing '*New stage*'. 2. 'Stage name' section with a text input field containing the value 'prod'. 3. 'Deployment description' section with a large text area. A light blue informational box is positioned between the 'Stage name' and 'Deployment description' sections, containing an information icon and the text: 'A new stage will be created with the default settings. Edit your stage settings on the **Stage** page.' At the bottom right of the modal, there are two buttons: 'Cancel' and 'Deploy'.



API Documentation

For my project's extension, I am writing API documentation because it is crucial for other developers to know how to use the API. Otherwise, if the developers did not have instructions, the API might not get used at all. You can do this writing documentation in the API Gateway console.

Once I prepared my documentation, I can publish it to the prod stage. You have to publish your API to a specific stage because different stages have different versions of the same API, so different explanations or requirements would be handy.

My published and downloaded documentation showed me both my manually written work and some automatically generated documentation from API Gateway. We can upload this into tools like Swagger and Redoc to generate beautiful web pages about my API.



```
},
"x-amazon-apigateway-documentation" : {
  "version" : "1",
  "createdDate" : "2026-08-20T22:20:48Z",
  "documentationParts" : [ {
    "location" : {
      "type" : "API"
    },
    "properties" : {
      "description" : "The UserRequestAPI manages user data requests. It supports operations to get user details based on u",
      "baseUrl" : "https://8btuodpzw2.execute-api.us-east-1.amazonaws.com/prod"
    }
  } ]
},
"x-amazon-apigateway-security-policy" : "TLS_1_0"
```



nextwork.ai

The place to learn & showcase your skills

Check out nextwork.ai for more projects

